Defiende el servidor

Versionado de código, datos, experimentos y modelos · MLOps 2026-II · Universidad Externado de Colombia · Allison Loango · Grupo A · Docente: Julián Zuluaga · 30 de octubre de 2026

Construimos un clasificador binario de conexiones de red (UNSW-NB15; 1 = ataque, clase positiva) y, sobre todo, su trazabilidad: cada versión del modelo se identifica por un commit Git, una versión DVC de los datos, un run de MLflow y una revisión SHA de Hugging Face, enlazados por el SHA-256 del artefacto. La versión final (v2) detecta 95,3 % de los ataques con FPR 4,8 % en validación; en test detecta 96,8 % pero su FPR sube a 17,5 %, hallazgo que se analiza en la sección 6.

1. Datos y auditoría

Se usaron las particiones publicadas, descargadas el 2026-10-30 del mirror fijado por commit que indica el enunciado (fuente oficial: UNSW Canberra). Ambos SHA-256 coinciden con el anexo. No hay faltantes. El 42 % de las filas de entrenamiento repite exactamente las 42 variables predictoras de otra fila, y hay grupos de entradas idénticas con etiquetas opuestas. Se excluyen label, attack_cat (revela la respuesta) e id; quedan 3 categóricas (proto, service, state) y 39 numéricas.

Archivo	Filas × col.	Normal / ataque	Filas duplicadas (entradas)	Grupos contradictorios (filas)	SHA-256
training-set	175.341 × 45	56.000 / 119.341	74.301	229 (940)	`bec7dd5ec88dc2a0…`
testing-set	82.332 × 45	37.000 / 45.332	28.386	6 (14)	`734fe6642edf758f…`

Política. Duplicados y contradicciones se conservan: reflejan la frecuencia real del tráfico y un ruido de etiqueta que también existe en test, así que eliminarlos inflaría la validación. Para que no haya fuga, cada grupo de entradas idénticas (hash de las 42 variables) va completo a entrenamiento o a validación mediante StratifiedGroupKFold (5 pliegues, semilla 42, pliegue 0 = validación): train 140.274 filas, validación 35.067 (68,1 % ataques en ambas), 0 grupos compartidos. Test solo se auditó: 8.541 filas (10,4 %) repiten entradas vistas, y state trae dos categorías nuevas (ACC, CLO), absorbidas por los codificadores.

2. Metodología y trazabilidad

Preprocesamiento y estimador viven en un único Pipeline de scikit-learn ajustado solo con entrenamiento (imputación, codificación, escalado), guardado como model.joblib. Los cuatro experimentos comparten partición, semilla y código de evaluación. Antes de entrenar, src/entrenar.py exige que código y configuración estén confirmados en Git; cada run de MLflow guarda hipótesis, cambio, parámetros, semilla, commit, md5 DVC de datos originales y preparados, métricas, scores de validación y el modelo completo con su SHA-256. DVC versiona los CSV recibidos (data/raw.dvc) y las particiones (data/prepared.dvc) en commits separados; su almacenamiento es un dataset público de Hugging Face accesible por HTTP sin credenciales. Hugging Face guarda v1 y v2 como dos commits del mismo repositorio, con tags.

Errores priorizados. Para un equipo de seguridad un ataque no detectado (FN) es el error más caro, pero cada falso positivo consume tiempo de analista y su exceso produce fatiga de alertas. La regla, fijada antes de ver resultados (configs/seleccion.yaml), es: maximizar el recall de ataque con FPR ≤ 5 % en validación, desempate por macro-F1. Precision, recall, FPR y macro-F1 se calculan desde la matriz de confusión; un denominador cero se reporta como 0 y se marca (no ocurrió en ninguna evaluación).

3. Cuatro experimentos (validación)

ID	Modelo · preprocesamiento	Umbral	Recall	Precision	FPR	Macro-F1	Run ID
E1	LogisticRegression · num=estandar; cat=onehot	0,50	0,9876	0,9230	0,1755	0,9225	`48de31fc1e57…`
		0,78	0,8950	0,9746	0,0496	0,9037	
E2	LogisticRegression · num=log1p_estandar; cat=onehot	0,50	0,9760	0,9385	0,1363	0,9295	`4f70fc75168a…`
		0,78	0,9132	0,9754	0,0491	0,9168	
E3	DecisionTree · num=ninguno; cat=ordinal	0,50	0,9772	0,9479	0,1146	0,9390	`d7c8bfee8d2f…`
		0,70	0,9325	0,9762	0,0485	0,9310	
E4	HistGradientBoosting · num=ninguno; cat=ordinal	0,50	0,9777	0,9611	0,0843	0,9511	`68d9aeb74e0d…`
		0,66	0,9532	0,9769	0,0480	0,9464	

Cada experimento aparece a 0,50 y a su umbral operativo (máximo recall con FPR ≤ 5 %). Fila resaltada: punto elegido para v2. Hipótesis, parámetros, commits y md5 DVC completos en evidencias/experimentos.csv.

ID	Hipótesis (resumen)	Resultado
E1	Referencia lineal: recall alto pero FPR alta por las colas largas de bytes/cargas.	Confirmada: FPR 17,6 % a 0,50; con FPR ≤ 5 % el recall cae a 89,5 %.
E2	log1p antes de estandarizar vuelve más lineal la señal y baja la FPR.	Parcial: FPR 13,6 % a 0,50 y recall 91,3 % con FPR ≤ 5 %; Fuzzers solo 54 %.
E3	Un árbol captura reglas por umbrales e interacciones (TTL, estado, servicio).	Confirmada: recall 93,2 % con FPR ≤ 5 %, supera a ambas regresiones.
E4	Un ensemble boosting reduce la varianza del árbol y da la mejor curva recall–FPR.	Confirmada: recall 95,3 % con FPR 4,8 %; errores concentrados en Fuzzers y Analysis.

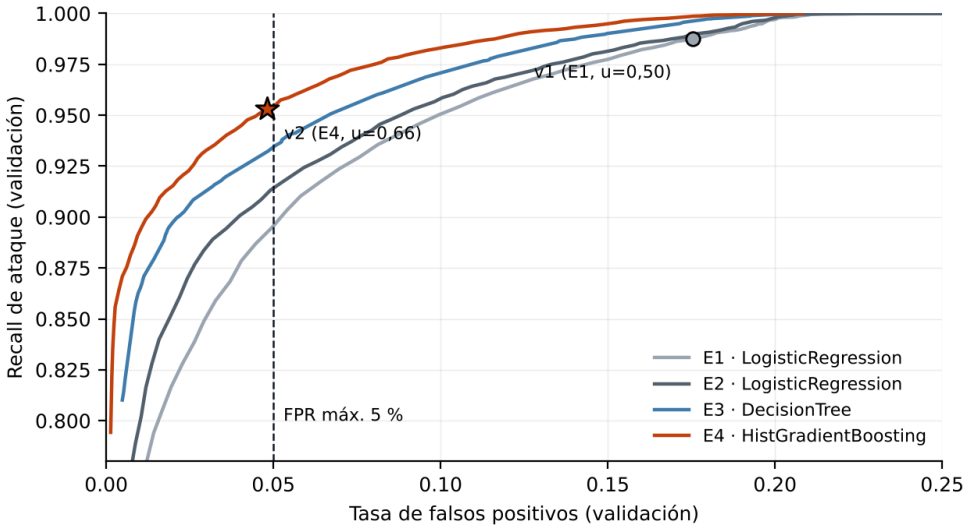


Figura 1. Curvas recall–FPR en validación. La línea discontinua marca el presupuesto de falsas alarmas; v1 queda muy a la derecha y v2 es el punto con mayor recall dentro de él.

4. Dos versiones: v1 y v2

v1 es la primera solución funcional (E1 con el umbral convencional 0,50), empaquetada y etiquetada antes de correr E2–E4. v2 cambia el modelo (HistGradientBoosting con categorías nativas, sin escalado) y el umbral (0,66), ambos elegidos con la regla de la sección 2. v2 no mejora todo: reduce las falsas alarmas a menos de un tercio y sube precision y macro-F1, a costa de 821 ataques no detectados más, sobre todo Fuzzers, cuyo tráfico se parece al normal. Aceptamos ese intercambio porque v1 alertaría sobre 1 de cada 6 conexiones normales, un volumen inmanejable para un SOC.

	Modelo	Umbral	Recall	Precision	FPR	Macro-F1	FP	FN	SHA-256 del artefacto
v1	Regresión logística (E1)	0,50	0,9876	0,9230	0,1755	0,9225	1.966	296	`36648d8096669151…`
v2	HistGradientBoosting (E4)	0,66	0,9532	0,9769	0,0480	0,9464	538	1.117	`23c42b34e7729858…`

Eslabón	v1	v2
Commit que entrenó (Git)	`720e330562248d9b1b5c68a4c647d12aad1abfea`	`0bf39dff1c2f7858f905ee8213512ce872f6b671`
Tag Git	v1	v2
Datos DVC (raw / prepared md5)	`fe93051ae2… / 5ae30cc8b0…`	`fe93051ae2… / 5ae30cc8b0…`
Run MLflow	`48de31fc1e57432194b41f4bc9e4a3a5`	`68d9aeb74e0d402d9209c10ca1a735da`
Revisión Hugging Face	`ver configs/referencias.json`	`ver configs/referencias.json`

Cadena de trazabilidad. Cada revisión de Hugging Face contiene model.joblib, config.json (umbral, variables, referencias), predict.py, requirements.txt y la model card.

5. Elección del umbral

Para cada run se barrieron umbrales de 0,01 a 0,99 sobre los scores de validación y se tomó el de mayor recall con FPR ≤ 5 %. Luego se compararon esos puntos entre experimentos; E4 a 0,66 gana en recall y también en macro-F1. Subir el umbral de 0,50 a 0,66 en E4 baja la FPR de 8,4 % a 4,8 % y cuesta 2,5 puntos porcentuales de recall. La decisión se escribió en evidencias/seleccion.json y se confirmó en Git (tag v2) antes del commit que ejecuta src/evaluar_test.py; el script verifica SHA-256 y umbral congelados y se niega a correr dos veces. Como el umbral se ajustó en validación, las métricas de validación al umbral elegido son algo optimistas.

6. Evaluación final en test

v2 se evaluó una sola vez sobre las 82.332 filas de test (sin eliminar ninguna): recall 0,9681, precision 0,8712, FPR 0,1754, macro-F1 0,9010, accuracy 0,9036. Las predicciones por fila (row_id, y_true, score_attack, y_pred) permiten recalcular estas cifras. El resultado empeora frente a validación y se conserva: el recall se mantiene, pero la FPR se multiplica por 3,7. La figura 3 muestra que el problema está en la clase normal y no en los ataques: el tráfico normal del archivo de test se distribuye distinto del de entrenamiento, y una validación extraída del mismo archivo de entrenamiento no podía anticiparlo. Con datos de operación, el umbral debería recalibrarse sobre tráfico normal reciente y no sobre test.

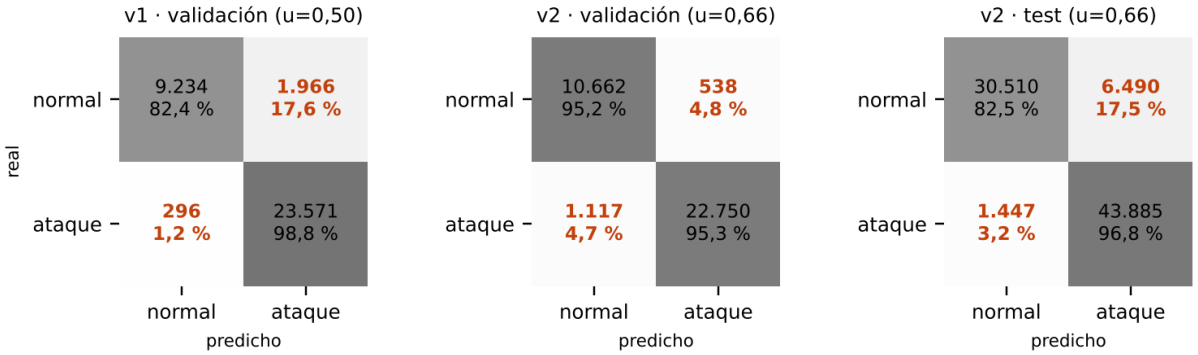


Figura 2. Matrices de confusión (en naranja, los errores con su tasa por fila real).

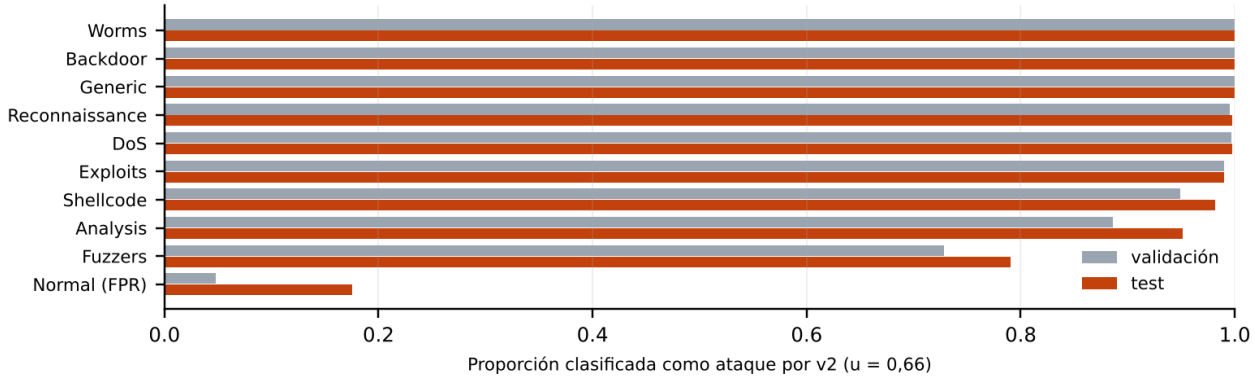


Figura 3. Proporción clasificada como ataque por categoría. En Normal esa proporción es la FPR.

7. Prueba de recuperación

scripts/recuperar.sh clona el código desde GitHub, crea un entorno nuevo con cachés vacías, ejecuta dvc pull, verifica huellas, descarga v1 y v2 por SHA y compara sus predicciones sobre 20 filas fijas de validación (tolerancia 1e-9). Un ensayo previo en contenedor limpio, con GitHub y Hugging Face simulados localmente, coincidió en 20/20 filas para ambas versiones (evidencias/recuperacion/ensayo_contenedor/). La prueba oficial queda en evidencias/recuperacion/entorno_nuevo/.

8. Limitaciones y conclusiones

UNSW-NB15 es tráfico de laboratorio de 2015: estos resultados no demuestran eficacia en una red actual. La validación proviene del archivo de entrenamiento, por lo que subestimó el cambio de distribución de test; el 10,4 % de test repite entradas ya vistas. Los grupos contradictorios son ruido irreducible. El artefacto es un pickle de scikit-learn 1.9.1: depende de esa versión y solo se carga tras verificar su SHA-256. Se usó todo el entrenamiento (sin muestreo) en CPU.

La respuesta a la pregunta central es sí: desde GitHub, el remote DVC y Hugging Face se recupera cualquiera de las dos versiones, se sabe de qué datos (md5 DVC) y experimento (run ID) viene, y produce exactamente las mismas predicciones. En lo metodológico, la regla fijada de antemano llevó a una v2 con un tercio de las falsas alarmas de v1 en validación; el test mostró que ese presupuesto no se transfiere a otra distribución de tráfico normal, y lo reportamos tal cual.

Referencias: Moustafa, N. & Slay, J. (2015). UNSW-NB15: a comprehensive data set for network intrusion detection systems. MilCIS, IEEE. · Pedregosa et al. (2011). Scikit-learn. JMLR 12. · Documentación oficial de Git, DVC, MLflow Tracking y Hugging Face Hub (descarga por revisión, model cards).